

Technical Assignment: AI-Powered Chinese Handwriting Grading App (PWA)

Instructions & Guidelines *Read First!!!*

1. Role & Scope

This assignment is intended for **both Front-End and Back-End developers** (the client is hiring for two roles). Candidates are expected to demonstrate proficiency across the full stack, but individual strengths will be taken into consideration.

2. Evaluation Focus

The client will primarily evaluate:

- **Features of the app and the communication to the backend.** i.e., *Take a photo, send to backend, have a simple word correction, send back to frontend for overlay of correct word in red pen. The flow* is important.
- **Note:** The accuracy of word recognition is **not** important for this task.

3. Minimum Requirements

Candidates are **encouraged, but not required, to complete every single task**. They **do what they can accomplish within the time given**. The client understands the scope is broad and values a well-executed partial solution over a rushed, incomplete one.

4. Tools & Platforms

Candidates **can use any tools**. They are not strictly required to use the exact tools listed (v0/Bolt, Supabase, Gemini, Vercel). Equivalent substitutions are acceptable as long as the core functionality remains identical.

5. Deadline

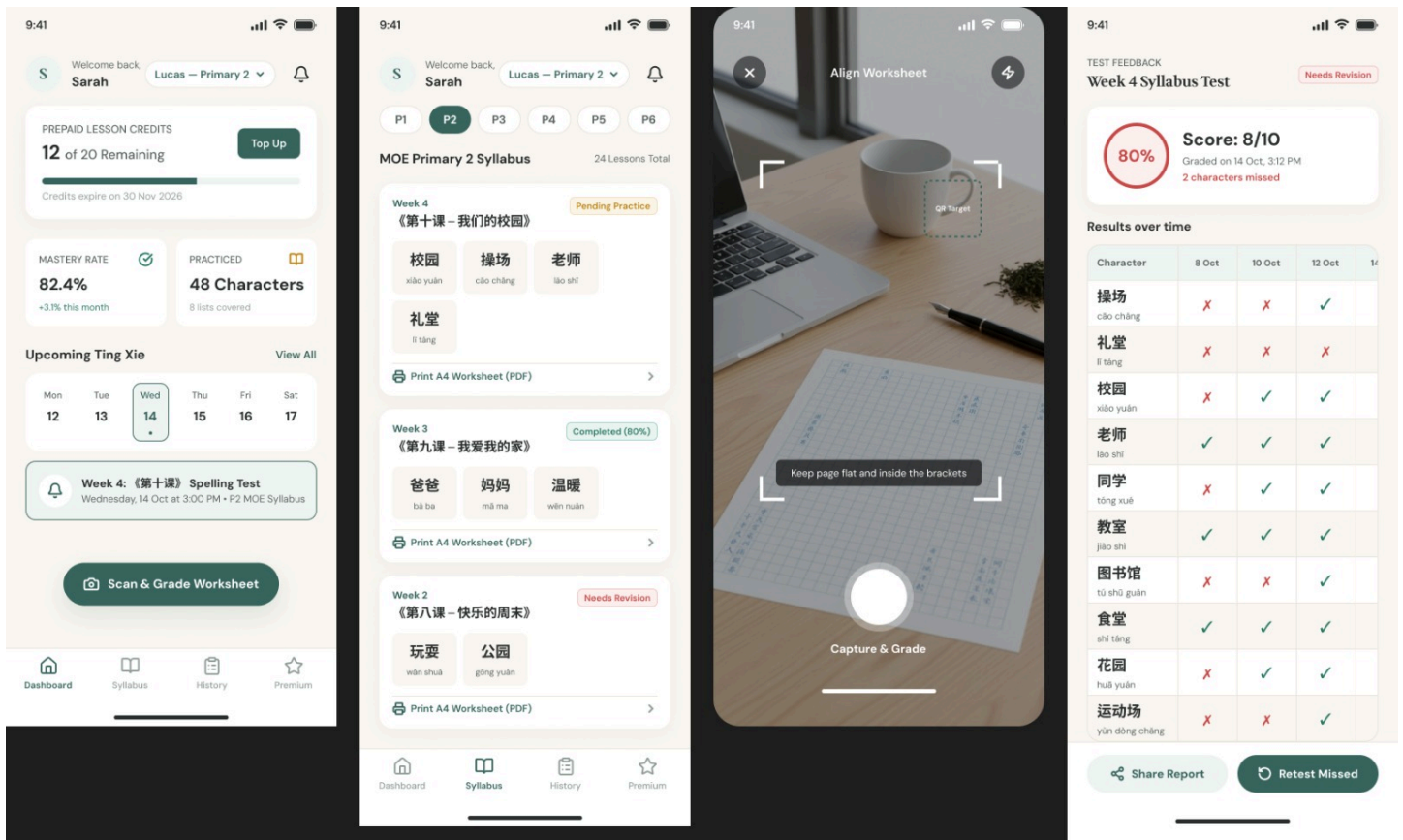
The assignment is due **5 days from the issued date**. The submission **timestamp will be verified** by checking the last updated date on GitHub.

6. Submission Format

Submit your work via **GitHub**. Include a live deployment URL in your repository's README.

UI Mockup References

The client has supplied visual mockup images that combine the layouts for Screens 1, 2, and 4. Your implementation should closely match these designs.



(**Note:** These images are for UI layout guidance; match the visual styling, spacing, and component hierarchy as closely as possible.)

Technical Requirements (Screens 1–5)

1. Front-End UI Generation (Screens 1 & 2)

- **Dashboard (Screen 1):**
 - Use v0/Bolt to convert the layout into Next.js/Tailwind CSS components.
 - Hardcode student profile details (Lucas - Primary 2).
 - Display the prepaid credits card (12/20 Remaining).
 - Include the weekly calendar strip.
- **Syllabus (Screen 2):**
 - Build the P1–P6 tab selector pills.
 - Create expandable MOE lesson cards showing vocabulary lists (e.g., 《第十课 - 我们的校园》).
 - Hardcode status tags (Pending, Completed) for the lesson cards.
- **PWA Setup:**
 - Add basic web app manifest settings (icons and viewport configuration) so the app installs seamlessly on mobile home screens.

2. Camera Capture & Live Alignment Viewfinder (Screen 3)

- **Camera API:**
 - Implement the HTML5 Media Devices API (`navigator.mediaDevices.getUserMedia`) to launch the native rear camera.
- **UI Overlay:**
 - Position a CSS absolute overlay over the camera feed.
 - Design elements include: a centered rectangular box with corner guidelines, instructions ("Keep page flat..."), a QR code target box, and a flash toggle button.
- **Capture Trigger:**
 - Program the shutter button to snap a high-res frame from the video stream, convert it into a BLOB image file, and trigger an uploading state.

3. Back-End Database & Image Storage Pipeline

- **Supabase Setup:**
 - Create basic PostgreSQL database tables:
 - `lessons` : Lesson titles, MOE level, word lists.
 - `submissions` : Student ID, timestamp, uploaded image URL, total score.
 - `character_results` : Foreign key to submission, character name, status (correct vs incorrect).
- **Upload Endpoint:**
 - Build an API route (POST /api/upload) to upload captured photos directly into a Supabase Storage Bucket and save the pending submission record.

4. AI Grading Engine Integration (Gemini 1.5 Flash)

- **Vision Pipeline:**
 - Connect your upload API to the Gemini 1.5 Flash API.
- **Prompt Engineering:**
 - Send the uploaded worksheet image to Gemini with a strict prompt:
"Compare the handwriting in this Tian Zige grid against the expected spelling list [操场, 礼堂, 老师]. Return a JSON array detailing which words were written correctly or incorrectly."
- **Score Processing & Feedback Overlay:**

- Receive the AI JSON payload, automatically compute the percentage score (e.g., 8/10 - 80%), and write the individual character results into your `character_results` database table.

(The flow—sending back to the front end for overlay of the correct word in red pen—is the key evaluation point.)

5. Results Matrix, Score Header & Deployment (Screen 4)

- **Score Overview Header:**

- Build the dynamic score display (Score: 8/10, status badge, date badge) pulling live data from the database.

- **Historical Matrix Table:**

- Build the result matrix table where rows represent tested Chinese words and columns represent test dates.
- Map green ticks (✓) and red crosses (✗) dynamically based on past records in Supabase.

- **Deployment:**

- Deploy the frontend and backend API to Vercel and deliver a live shareable URL.

Good luck, and happy coding!